Predictive Weather Guidance With Route Engine

Nimish M Biju
Riya Roy
Nandhakrishna V S
Pavithra R Raj
Nihal Mohamed Shafi

Introduction

- Navigation apps prioritize shortest distance or fastest time
- Weather (rain, fog, wind, storms) impacts safety, comfort, and reliability
- Users must switch between map and weather apps
- Integrating both enables smarter and safer travel planning

Problem Statement

- Route planners don’t treat weather as a key decision factor
- Users must manually assess weather impact on routes
- Weather varies along the route; origin-only data is insufficient
- Need a simple, beginner-friendly system combining routing, weather, and visualization

Aim of the Project

- Develop a web app that generates routes with weather conditions along the path
- Enable users to make informed decisions before starting their journey

Objectives

- Build an interactive map-based web interface
- Generate routes between source and destination
- Extract key points (start, midpoint, destination)
- Fetch weather data for these points via API
- Display route and weather together clearly

Literature Review

Route Optimization Considering Environmental Factors

Authors: L. Zhang et al.

- Proposed route optimization including environmental factors like rainfall and wind.
- Applied data analysis techniques to improve travel safety and reliability.

Limitation:

- Complex model not suitable for beginner-friendly or lightweight web applications.

Web-Based Travel Planning System with Visualization

Authors: H. Rajora et al.

- Developed a web-based system integrating data processing, prediction, and visualization.
- Used machine learning and APIs to present user-friendly outputs.

Limitation:

- Focuses on prediction systems rather than real-time route-weather integration.

Literature Review

Smart Navigation System with Weather Integration

Authors: R. Kumar et al.

- Developed a web-based navigation system combining route generation and weather forecasting.
- Used API-based data fetching to display weather at source and destination.

Limitation:

- Considers weather only at endpoints, ignoring variations along the route.

Weather-Aware Route Planning Using Web Technologies

Authors: J. Smith et al.

- Proposed a route planning system integrating navigation with real-time weather data.
- Utilized mapping APIs and weather APIs to visualize conditions along travel routes.

Limitation:

- Limited to static checkpoints; does not dynamically adjust routes based on weather changes.

Literature Review

Weather Forecast Integration in Navigation Systems

Authors: M. Lee et al.

- Proposed integrating weather forecast data into navigation interfaces for better decision-making.
- Highlighted importance of real-time data in improving travel safety.

Limitation:

- Lacks detailed visualization of weather conditions along the entire route.

Proposed System

The user enters a start point , start time and destination in the web application.

The system generates a route on an interactive map.

The route geometry is sampled at key points such as origin, midpoint, and destination.

The application requests weather data for these coordinates.

Weather details are displayed beside the route together with simple warnings.

The first version acts as a decision-support tool rather than a full automatic rerouting engine.

User Input	**Routing**	Weather Retrieval	Processing	Visualization

Dataset

Primary data sources

- OpenStreetMap – road network and base map

https://www.openstreetmap.org/

- Nominatim – geocoding and reverse geocoding

https://nominatim.org/

- OSRM – route computation over OSM road network

https://project-osrm.org/

- OpenWeather – forecast data by coordinates

https://openweathermap.org/forecast5

- Kerala/India support:

https://wiki.openstreetmap.org/wiki/Kerala

https://download.geofabrik.de/asia/india.html

Technology Stack

Frontend and development

HTML5, CSS3, and JavaScript.

VS Code and Live Server for local development.

Optional GitHub for version control.

Core services and libraries

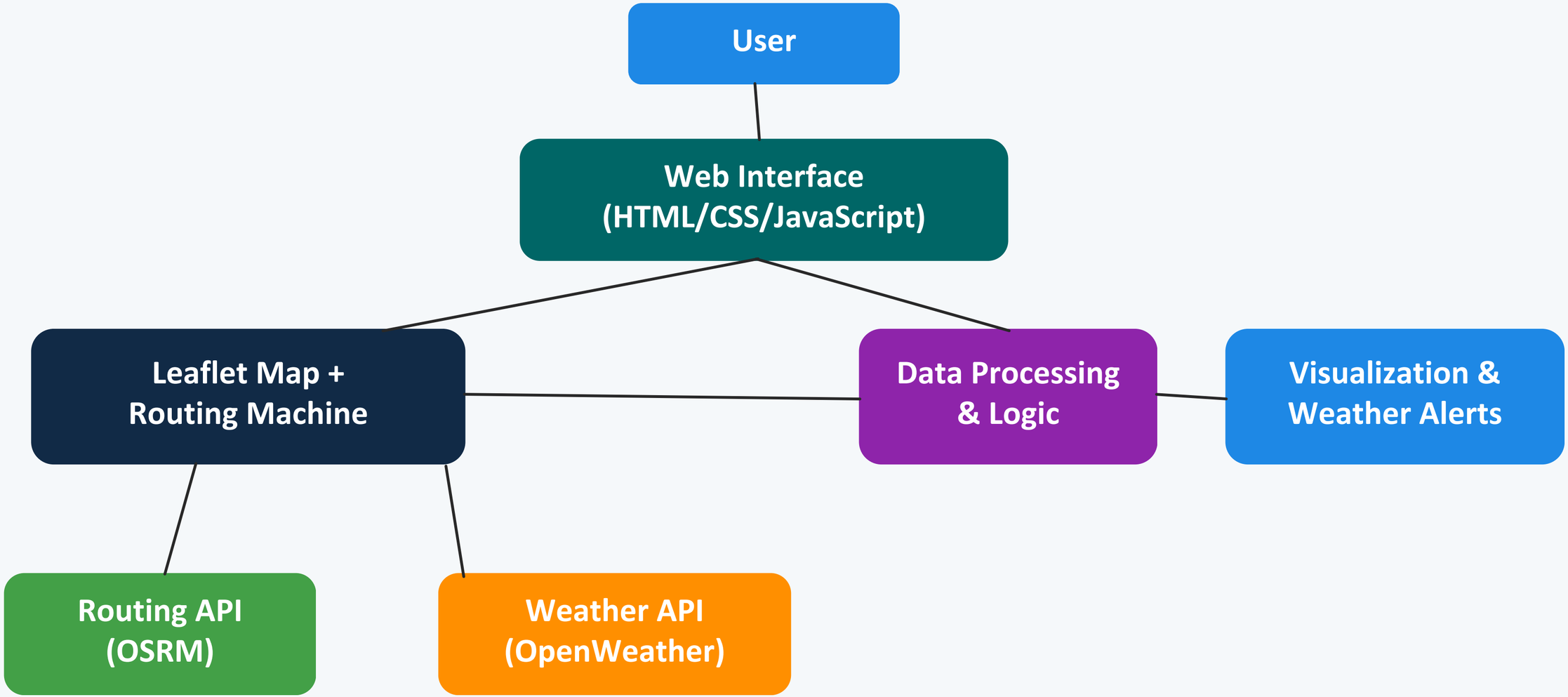
Leaflet for interactive map display.

Leaflet Routing Machine for routing interface.

OSRM-compatible backend for route generation.

OpenWeather API for current weather and forecasts.

System Architecture



Pseudocode- Road Mapping & Path Computation Algorithm

1. Start the algorithm.
2. Read the input start location text and destination location text.
3. Convert both location texts into geographic coordinates using geocoding.
4. Check if either the start or destination coordinates are null.
5. If any coordinate is null, return "Invalid start or destination" and terminate the algorithm.
6. Request the route from the OSRM service using the obtained coordinates.
7. Check if the route response is empty.
8. If the response is empty, return "Route not found" and terminate the algorithm.
9. Extract the route path coordinates, total distance, and total travel time from the response.
10. Compute the midpoint index as half of the length of the route path.
11. Determine the midpoint coordinate using the midpoint index.
12. Draw the route path on the map.
13. Return the route path, total distance, total travel time, and midpoint coordinate.
14. End the algorithm.

Pseudocode- Weather Forecast Extraction & API Integration Algorithm

1. Start the algorithm.
2. Read the start and destination coordinates.
3. Initialize a list with all coordinates.
4. For each coordinate, construct the API URL using latitude and longitude.
5. Fetch forecast data from the OpenWeather API.
6. If the response is empty, store "Forecast unavailable"; otherwise extract and store the forecast data.
7. Assign forecast data to start, midpoint, and destination variables.
8. Return all forecast data.
9. End the algorithm.

Pseudocode- Route–Weather Synchronisation & Timeline Generation Algorithm

1. Start the algorithm.
2. Read departure time, total travel time, and forecast data (start, midpoint, destination).
3. Compute arrival times:
4. Start = departure time
5. Midpoint = departure time + (total travel time / 2)
6. Destination = departure time + total travel time
7. For each location (start, midpoint, destination):
8. a. Initialize minimum time difference to infinity and closest forecast to null.
9. b. For each forecast entry, compute the absolute time difference from arrival time.
10. c. If the difference is smaller, update the closest forecast.
11. d. Store the closest forecast for that location.
12. Return matched forecasts for start, midpoint, and destination.
13. End the algorithm.

Pseudocode- Backend Orchestration & API Delivery Algorithm

1. Start the algorithm.
2. Read start location, destination location, and departure time.
3. Geocode locations to obtain coordinates.
4. If coordinates are invalid, return "Location not found" and terminate.
5. Request route from OSRM; if unavailable, return "Route unavailable" and terminate.
6. Extract route path, distance, and travel time; compute midpoint and its place name.
7. Fetch weather forecast data for start, midpoint, and destination.
8. Compute arrival times based on departure time and travel duration.
9. Match closest forecast entries for each location.
10. Analyze forecasts to generate weather warnings.
11. Construct and return structured route-weather response.
12. End the algorithm.

Pseudocode- Frontend Visualisation & UI Algorithm

1. Start the algorithm.
2. While the user types in the search box, fetch and display location suggestions after a short delay.
3. When the user selects start and destination and initiates the request, send the inputs to the backend.
4. Receive the route-weather response.
5. If the response contains an error, display the error message and terminate.
6. Clear previous UI elements.
7. Render the route on the map and adjust the view.
8. Display route summary details (locations, midpoint, distance, travel time).
9. Display weather information for start, midpoint, and destination.
10. If warnings exist, display them prominently.
11. End the algorithm.

Backend Technologies & API Overview

Core Technologies

Node.js – Server-side execution

Express.js – REST API & request handling

CORS Middleware – Enables frontend–backend communication

File System (fs) – JSON-based storage (.database.json, .users.json)

bcryptjs – Secure password hashing

svg-captcha – CAPTCHA generation

API Endpoints

GET /api/history – Retrieve stored route data

POST /api/history – Save route with timestamp

DELETE /api/history – Clear records (admin only)

GET /api/captcha – Generate CAPTCHA { id, svg }

POST /api/login – Validate user + CAPTCHA

POST /api/register – Register user (hashed password)

CAPTCHA & Password Security

CAPTCHA System

- Generated using svg-captcha (distorted SVG)
- Returns { id, svg }; stored in memory (expires in 5 min)

Working:

Generate → Display → User Input → Verify → Delete (one-time use)

Purpose:

- Prevent bots (OCR-resistant distortions)
- Ensure human verification

Password Handling

Frontend Validation:

- ≥ 8 characters, uppercase, lowercase, number, special character
- Real-time strength check

Backend Security:

- Hashed using bcryptjs (cost: 10)
- Stored securely (no plain text)

Login Flow:

CAPTCHA → Password check (bcrypt.compareSync()) → Access control

Workflow

1. User enters source and destination.

2. The system geocodes the locations and generates a route.

3. Route coordinates are extracted from the routing response.

4. Key points on the route are selected for sampling.

5. Weather API calls are made for the selected coordinates.

6. Weather summaries and simple alerts are displayed to the user.

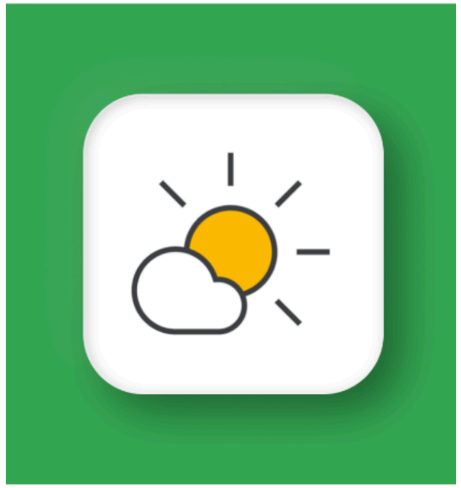
Future scope

- Real-time Weather Updates: Integrate live weather APIs for continuous updates during travel.
- Dynamic Route Adjustment: Suggest alternate routes based on changing weather or traffic conditions.
- Traffic Data Integration: Combine weather insights with real-time traffic information for better planning.
- Multi-Stop Route Planning: Allow users to add multiple stops and analyze weather at each point.
- Mobile Application Development: Extend the system into a dedicated mobile app for accessibility.
- User Personalization: Provide customized alerts based on user preferences (e.g., avoid rain or fog).
- Predictive Analytics: Use machine learning to forecast potential travel risks more accurately.
- Voice Assistance Integration: Enable voice-based route planning and weather updates.

References

OpenStreetMap, OpenStreetMap Data : **https://www.openstreetmap.org/**
OSRM, Routing API Documentation : **http://project-osrm.org/**
OpenWeather, Weather API Documentation : **https://openweathermap.org/ap**i
Nominatim, Geocoding API Documentation: **https://nominatim.org/**
Leaflet, Interactive Maps Documentation : **https://leafletjs.com/**
REST API, Architectural Style for Networked Applications.

Publications



Planning a trip? In this blog post, I'll show you how to use the Routes API and Google Weather API to build a Weather Along Route app that provides real time weather updates for every stop on your journey. Perfect for planning road trips, delivery routes, or for everyday travel.

https://blog.afi.io/blog/google-maps-weather-along-route-weather-aware-planner/#:~:text=Google%20Maps%20weather%20along%20route,in%20the%20high%20mountain%20passes

Conclusion

The proposed system combines route planning and weather information in a single web interface.

It addresses a practical user need and is suitable as a beginner mini project.

The application is simple, scalable, and can be extended to real-time alerts, route risk scoring, and weather-based rerouting.

The project also demonstrates integration of geospatial APIs, web technologies, and environmental data.

Thank you